

Lecture 8 : Class and Objects

지금까지 우리가 코드를 짜온 방식

데이터 보호 불가

함수는 데이터를 신뢰할 수 없다. `get_average()`는 딕셔너리가 올바른 형식인지 알 방법이 없다. 외부에서 잘못된 상태를 주입하는 것을 막을 울타리가 없다.

확장성 부재

같은 구조, 끝없는 반복. 학생이 늘어날수록 동일한 구조의 딕셔너리를 반복 생성해야 하며, 구조가 변경될 시 관련된 모든 코드를 수정해야 하는 관리 지옥에 빠진다.

데이터와 로직이 따로 노는 구조

절차적 프로그래밍에서 데이터 오염이 일어남

학생 데이터 — 딕셔너리로 관리

```
student_1 = {"name": "Alice", "mid": 85, "final": 90}
```

```
student_2 = {"name": "Bob", "mid": 70, "final": 65}
```

성적 계산 — 데이터와 분리된 함수

```
def get_average(student):  
    return (student["mid"] + student["final"]) / 2
```

아무도 이 실수를 막아주지 않는다 ↓

```
student_1["mid"] = "A+" # 정상 실행됨 (데이터 오염)
```

```
get_average(student_1) # → TypeError 발생
```

데이터(State)와 그것을 다루는 함수(Behavior)가 **서로를 모른 채 분리되어** 있습니다. 이 둘을 하나의 **안전한 단위 (Object)**로 묶어 **데이터 스스로를 보호**하게 만드는 것이 OOP의 핵심입니다.

객체

상태(State): 객체가 가진 고유한 데이터

e.g. 자동차의 현재 속도, 배기량

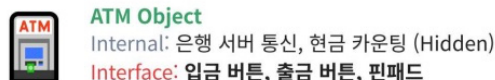
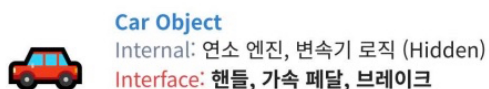
행동(Behavior): 상태를 가지고 외부에서 액션을 취할 수 있는 결합된 형태

e.g. 브레이크 밟기, 기어 변속

추상화(Abstraction): 복잡한 내부를 숨기고 필요한 기능(인터페이스)만 노출

왜 추상화할까?

외부의 로직은 몰라도 **인터페이스(Interface)**만 알고 있으면 이걸 사용하는 데 아무런 문제가 없다



추상화는 사용자에게 **"무엇을 할 수 있는지"**만 보여주고, **"어떻게 동작하는지"**는 감추는 것입니다. 이로 인해 코드의 복잡성이 **획기적으로 줄어들고**, 데이터 오염으로부터 안전해짐.

클래스와 인스턴스

클래스 : 설계도. 객체가 가져야 할 속성과 행동을 정의한 틀. 메모리를 차지하지 않는 추상적인 개념.

인스턴스 : 실제. 설계도로부터 찍어내어 메모리에 실제로 할당된 데이터. 각 인스턴스는 독립적인 상태를 유지.

클래스는 함수(Methods)와 데이터(Attributes)를 하나의 바구니에 담는 틀입니다.

인스턴스는 이 틀을 통해 생성된 독립적인 메모리 객체입니다.

같은 문제, 다른 접근

Before Procedural 데이터, 함수가 분리된 관리 방식

```
student_1 = {"name": "Alice", "mid": 85, "final": 90}
student_2 = {"name": "Bob", "mid": 70, "final": 65}
def get_average(student):
    return (student["mid"] + student["final"]) / 2
#아무도 막지 않는다
student_1["mid"] = "A+"
get_average(student_1) # TypeError
```

AFTER -OOP 데이터와 메서드를 하나의 단위로 묶기

```
class Student:
    def __init__(self, name, mid, final):
        #데이터 보호도 객체 안에서 해결
        if not isinstance(mid, int) or not isinstance(final, int):
            raise TypeError("점수는 정수여야 합니다")
        self.name = name
        self.mid = mid
        self.final = final
    def get_average(self):
        return (self.mid + self.final) / 2

s1 = Student("Alice", 85, 90)
s1.get_average() #✓ 데이터와 로직이 함께
```

Student 객체는 자신의 **데이터**(mid, final)와 그것을 다루는 **함수**(get_average)를 함께 소유한다. 데이터 **보호와 반복 구조** 관리, 두 균열 모두 설계도(class) 하나로 해소.

클래스의 기본 골격

```
class Student:
    def __init__(self, name, sid, email):
        self.name = name
        self.sid = sid
        self.email = email
    def greet(self):
        print(f"안녕하세요, {self.name}({self.sid})입니다.")
s1 = Student("Alice", "20260001", "alice@cau.ac.kr")
s1.greet()
```

#새로운 타입 선언 키워드&이름
#생성자. 객체가 생성되는 순간 자동 호출
#인스턴스 메서드.
#인스턴스 생성

self는 자기 참조. 모든 메서드의 첫 번째 인자. 호출된 객체 자신을 가리키며 파이썬이 자동 전달.

클래스 실습

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_available = True
    def checkout(self):
        self.is_available = False
    def return_book(self):
        self.is_available = True

if __name__ == "__main__":
    b1 = Book("Python", "Choi")
    print(f"대출 전 상태 (is_available): {b1.is_available}")
```

#1. 인자를 객체의 속성으로 저장
빈칸 1: 저자 속성 저장
빈칸 2: 기본 대출 가능 상태를 True로 설정

#2. 대출 상태를 False로 변경
빈칸 3: 대출 시 이용 가능 여부를 False로 변경

#3. 반납 상태를 True로 변경
빈칸 4: 반납 시 이용 가능 여부를 다시 True로 변경

하단 실습 가이드에 따른 객체 생성 및 테스트 로직
b1 인스턴스 생성 (책 제목: "Python", 저자: "Choi")
출력: True

```

b1.checkout()                # checkout() 메서드 호출
print(f"checkout() 호출 후 상태 (is_available): {b1.is_available}") # 출력: False

b1.return_book()             # return_book() 메서드 호출
print(f"return_book() 호출 후 상태 (is_available): {b1.is_available}") # 출력: True

```

self란?

- 호출 변환
`s.greet()` -> `Student.greet(s)` 파이썬이 자동으로 변환. `s`가 `self` 자리로 전달된다.
- ID()로 확인 가능
 인스턴스 밖에서 찍은 주소 == `self`가 가리키는 주소
- 인스턴스가 여럿일 때 `self`의 역할
`s1.greet()` #→ `Student.greet(s1)` : `self`= `s1` 주소 → "Alice" 출력
`s2.greet()` #→ `Student.greet(s2)` : `self` = `s2` 주소 → "Bob" 출력
`#greet()` 코드는 클래스에 하나뿐 - `self` 덕분에 "누구의" 메서드인지 구분된다

self는 호출된 인스턴스의 **메모리 주소를 가리키는 참조**다. 메

서드 코드는 클래스에 단 하나만 존재하지만, `self` 가 매번 다른 인스턴스를 가리키기 때문에 각 객체가 자신의 데이터로 동작할 수 있다.

객체의 독립성 Memory Isolation

객체지향 프로그래밍의 핵심은 **상태(State)를 안전하게 보호**하는 것

```

s1 = Student("Alice")        # 1. 두 개의 독립적인 인스턴스 생성
s2 = Student("Bob")

print(id(s1))                 # Output: 1407...8400 (Memory A)           # 2. 물리적 주소 확인 (id 함수)
print(id(s2))                 # Output: 1407...9216 (Memory B)

s1.name = "Alicia"           # 3. 데이터 수정 (Modification)

print(s1.name) # Alicia       # 4. 결과 확인: s2는 여전히 'Bob' 유지
print(s2.name) # Bob (격리 성공!)

```

★파이썬은 효율성을 위해 메서드(Behavior)는 공유하지만, 인스턴스 변수(State)는 철저히 개별 관리한다

오류 찾기!

```

class Dog:
    def _init_(self, name):      # ??? 언더바 오타. 언더바 2개 써야 한다.
        self.name = name

class Dog2:
    def bark():                  # ??? self 누락. self를 써 줘야 한다
        print("Woof!")

d1 = Dog2()
d1.bark()

```

```
class Book:
    def __init__(self, title, price):
        pass
```

b1 = Book("Python") # 생성자에서 argument의 개수를 2개로 했기 때문에 2개를 전달해야 함.

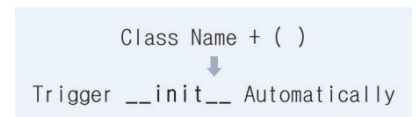
클래스와 객체의 상호작용

```
# STEP 1. 설계 단계 (Class Definition)                      '틀' 상태. 아직 메모리에 아무것도 존재하지 않음.
class Student:
    def __init__(self, n):
        self.name = n
```

```
# STEP 2. 실행 단계 (Object Creation)
s1 = Student("Alice")
```

설계도(Class) 안에 self라고 적어두는 이유는, 나중에 밖에서 실제로 만들어질 다양한 객체(s1, s2 등)들과 연결될 통로를 미리 만들어두기 위해서입니다.

왜 __init__을 직접 호출하지 않을까?



객체가 만들어지는 순간 "초기화는 필수"이므로, 파이썬이 이 호출 문법에 init 실행을 미리 포함시켜 둔 것입니다. 모든 것은 객체이다. my_str = "Python" 조차도 str클래스의 인스턴스를 만드는 과정이다. __init__을 일일이 부르지 않아도 파이썬의 모든 데이터가 만들어진다.

-> init을 따로 부르는 번거로움을 없애고, 생성과 동시에 데이터를 세팅하기 위해 호출 문법에 내장시킨 파이썬의 convention이다.

★ 호출과 동시에 초기화까지

모든 것은 객체

AA = 123456789 같은 간단한 할당문도 사실은 파이썬 내장 클래스를 호출하여 객체를 만드는 과정을 거친다. Integer Caching : 자주 쓰는 작은 숫자는 효율을 위해 미리 만들어두고, 큰 숫자나 실수는 필요할 때마다 호출

직접 만든 클래스나 파이썬 제공 클래스나 "이름을 호출하면 객체가 생성되고 초기화된다"

인스턴스 변수 vs 클래스 변수

```
class Student:
    def __init__(self, name):
        self.name = name                      # 각자의 이름                      인스턴스 변수

class Student:
    school = "Chung-Ang"                      # 공통 학교 (교수님이 언급하신 학교 이름 변수)                      클래스 변수

    def __init__(self, name):
        pass # ... 후속 로직 생략 표시
```

인스턴스 변수 : self 를 통해 인스턴스에 속함. 객체마다 다른 값을 가질 수 있음. 각 객체의 독립된 공간에 저장.
클래스 변수 : 메서드 밖, 클래스 바로 아래 정의. 모든 객체가 동일한 값을 공유. 클래스에 1개 존재.

변수 가리기(Shadowing)

```
class Cat:
    species = "Mammal"      # 클래스 변수

c1 = Cat()
c2 = Cat()

c1.species = "Alien"       # c1에만 새로운 인스턴스 변수 생성!

print(c1.species) # "Alien" (Shadowed)
print(c2.species) # "Mammal" (Still Class Var)
```

인스턴스를 통해 클래스 변수와 같은 이름의 변수를 수정하려고 하면, 의도치 않게 개별 변수가 생성되어 데이터 불일치가 발생할 수 있음.

클래스 변수가 인스턴스 변수에 의해 가려진다!

LAB 스마트 쇼핑물

```
class Product:
    discount_rate = 0.1 # 클래스 변수

    def __init__(self, name, price):
        self.name = name
        self.price = price

    def get_sale_price(self):
        return self.price * (1 - Product.discount_rate) # Product.discount_rate를 활용해 계산
```

클래스 변수: 할인율(discount_rate)을 0.1로 설정
인스턴스 변수: 제품명(name)과 가격(price) 초기화
메서드: get_sale_price() 구현(할인율 적용)

블프 할인하기! Global 정책을 변경할 수 있다.

Product.discount_rate = 0.3

객체 내부 보호

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance # 완전히 공개된 상태

    def deposit(self, amount):
        self.balance += amount

acc = Account("Alice", 1000)
# 아무도 막지 않는다 ↓
acc.balance = -999999 # 잔액이 음수?
acc.balance = "hello" # 문자열도 가능?
```

Account 클래스의 self.balance 속성은 외부에 완전히 노출되어 있다. 누구나 마음대로 접근해서 값을 뜯어 고칠 수 있는 상태.

캡슐화 (Encapsulation) : 데이터(상태)와 메서드(행동)를 하나로 묶고 내부를 감추는 것

정보 은닉 (Information Hiding) : 내부 구현을 외부로부터 철저히 차단하는 것.

내부 구현이 바뀌어도 외부에서는 영향을 받지 않음

해결책 : __private 변수 활용, 네임 망글링 (Name Mangling), getter / setter 인터페이스

Private Variable 프라이빗 변수

이름 앞에 __ 언더바 두 개 붙이기

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance # Private 변수

acc = Account("Alice", 1000)
```

1. 직접 접근 시도
print(acc.__balance)
AttributeError 발생!

2. 값 수정 시도
acc.__balance = -999999
print(acc.__balance) # **-999999**
수정된 것처럼 보인다? 그런데 원래 잔액은 어떻게 됐을까?

외부 접근 차단: 클래스 내부에서 관리하는 속성을 외부에서 마음대로 조작하면 프로그램에 문제가 생길 수 있습니다

안전장치 마련: 이를 막기 위해 변수 이름 앞에 __를 붙여 Private 변수로 선언함으로써 "이건 건드리지 마"라고 바깥에 신호를 주는 것

balance 변수는 클래스 외부에서 보이지 않는 상태가 됨.
코드에 직접 접근을 시도하면 AttributeError 발생

but, 수정을 시도하면 값이 수정된 것처럼 보이지만, **실제로는 새로운 변수가 생성되어버린 것임.**

-> __는 강력한 보안 장치가 아니다.

-> 데이터 접근에 필요함 통로 Getter/Setter가 필요하다.

Name Mangling 네임 망글링

```
acc = Account("Alice", 1000)
```

엉뚱한 이름으로 수정을 시도하면?
acc.__balance = -999999

객체의 실제 속성 지도를 출력
print(acc.__dict__)

출력 결과:
{
 'owner': 'Alice',
 '_Account__balance': 1000, # 원본 보존
 '__balance': -999999 # 별개로 추가됨
}

print(acc._Account__balance) # 1000

파이썬은 변수명 앞에 더블 언더스코어(__variable)가 붙으면 내부적으로 변수의 이름을 자동 변환
원래 이름 앞에 _클래스이름이 추가되어 _ClassName_variable이 된다.

바뀐 이름으로는 접근 가능. 따라서 Mangling은 실수 방지(이름 충돌)를 위한 기능이다.

Getter & Setter

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self._balance = balance

    def get_balance(self): # Getter
        return self._balance

    def set_balance(self, amount): # Setter
        if isinstance(amount, (int, float)) and amount >= 0:
            self._balance = amount
        else:
            raise ValueError("Invalid amount!")
```

Getter : 신뢰하고 데이터를 안전하게 꺼내갈 수 있는 메서드. 여기서는 `get_balance`
외부 코드가 객체 내부를 직접 건드리지 않고, 정해진 인터페이스를 통해서만 소통하도록 유도하는 것

Setter : 데이터를 수정하기 전에 설계자가 정한 올바른 데이터 규칙에 맞는지 확인하는 필터 역할.
코드의 검증 로직. 여기서는 `set_balance`를 통해 입력된 값이 정수나 실수 타입이 맞는지 확인.
데이터의 무결성 유지

언더스코어 사용법 정리 및 관례

<code>_variable</code>	Internal Use: 내부용 관례. 외부 접근은 가능하나 직접 수정을 자제하라는 의미.
<code>variable_</code>	Avoid Conflict: <code>class</code> , <code>list</code> 등 예약어와 이름 충돌 방지용.
<code>__attribute</code>	Name Mangling: 클래스 외부 접근 시 이름을 변형하여 속성을 보호 (정보 은닉).
<code>__str__</code> , <code>__add__</code>	Dunder Methods: 특정 상황에서 자동 호출되는 특수 메서드 (PART 4 연계).

이름 충돌 방지용

클래스에서 정보 은닉

특정 상황에서 자동 호출되는 특수 메서드

str vs repr

```
class Coordinate:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self): # print()용
        return f"({self.x}, {self.y})"

    def __repr__(self): # Debugging용
        return f"Coordinate({self.x}, {self.y})"
```

pos = Coordinate(10, 20)
`print(pos)` # 출력: (10, 20)
`repr(pos)` # 반환: 'Coordinate(10, 20)'

`__str__` : `print()`나 `str()` 호출 시 실행됨. 가독성 높은 정보 제공이 목표
`__repr__` : 개발자가 디버깅 시 명확한 정체성을 파악할 수 있도록 도움.

파이썬은 객체를 출력할 때 `__str__`이 없으면 `__repr__`을 대신 호출하여 어떻게든 정보를 보여주려고 노력
but, `__repr__`이 없고 `__str__`만 있는 상황에서는
최소한 `__repr__`만큼은 항상 구현해 두는 것이 좋다. 백업용으로

이걸 왜 해야 할까?

```

class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __repr__(self):
        return f"Book({self.title!r}, {self.author!r})"

    def __str__(self):
        return f"'{self.title}' by {self.author}"

b = Book("The Odyssey", "Homer")

print(b)    # 'The Odyssey' by Homer
print([b])  # [Book('The Odyssey', 'Homer')]
# 미정의 시: <__main__.Book object at 0x102...>

```

메서드가 없으면 클래스 이름과 메모리 주소(hex)를 출력한다.

`__repr__` 구현 시(필수) : 데이터 상태를 명확히 보여줌.

`__str__` 구현 시 (선택) : 최종 사용자에게 보여줄 친절한 문장을 완성함.

`!r` : 문자열이면 따옴표까지 포함해서 출력되게 하기.

```

그냥 출력할 때 :          print(b)          ->    __str__ 호출
리스트에 담아서 출력할 때 : print([b])       ->    __repr__ 호출

```

산술 연산자 오버로딩

```

class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        # 항상 새로운 Vector 객체를 생성하여 반환
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(2, 4)
v2 = Vector(1, 3)
v3 = v1 + v2      # v1.__add__(v2) 호출

```

`a+b` 라는 코드를 실행하면

내부적으로는 `a.__add__(b)`라는 매직 메서드 호출로 자동 변환

+	<code>__add__(self, other)</code>
-	<code>__sub__(self, other)</code>
*	<code>__mul__(self, other)</code>
/	<code>__truediv__(self, other)</code>
//	<code>__floordiv__(self, other)</code>

비교 연산자 오버로딩

```

class Rectangle:
    def __init__(self, w, h):
        self.area = w * h

    def __eq__(self, other):
        return self.area == other.area

    def __lt__(self, other):
        return self.area < other.area

```

```

r1 = Rectangle(10, 5)
r2 = Rectangle(25, 2)
print(r1 == r2)    # True (넓이 기준 동일)

```

==	<code>__eq__(self, other)</code>
!=	<code>__ne__(self, other)</code>
<	<code>__lt__(self, other)</code>
>	<code>__gt__(self, other)</code>
<=	<code>__le__(self, other)</code>

Lecture 9 : Inheritance and Polymorphism

상속이란? 메서드 오버라이딩 다형성 & object 클래스

일반 학생 (Student)

```
class Student:
    def __init__(self, name, id):
        self.name = name
        self.id = id

    def study(self):
        print(f'"{self.name}" 학생이 공부합니다.')
```

대학원생 (GradStudent)

```
class GradStudent:
    def __init__(self, name, id, lab):
        self.name = name
        self.id = id
        self.lab = lab

    def study(self): # 메서드 역시 중복!
        print(f'"{self.name}" 학생이 공부합니다.')
```

DRY(Don't Repeat Yourself) 위반 : 위반 코드의 재사용성이 떨어지고 관리가 힘들어짐.

-> 공통된 속성을 물려받을 수 있다면 어떨까?

상속 기본 문법

```
class ChildClass ( ParentClass ):
```

• Child (Derived/Sub):

부모의 유산을 물려받는 새로운 클래스

• Parent (Base/Super):

데이터와 기능을 제공하는 기존 클래스

class Student: # 부모 클래스 (이미 존재함)

```
    def study(self):
        print("공부 중입니다...")
```

class GradStudent(Student): # Student를 상속함

```
    pass # 아무 내용도 정의하지 않음!
```

--- 객체 생성 및 테스트 ---

```
student_g = GradStudent()
```

GradStudent에는 study가 없지만,

부모로부터 물려받았기 때문에 호출 가능!

```
student_g.study()
```

Output: 공부 중입니다...

물려받는 것 : Methods & Variables.

클래스 변수 + 메서드 + Public Members(외부 노출된 모든 요소)

만약 자식에서 __init__을 만든다면, 부모의 __init__은 완전히 덮혀씌워진다.

class Student:

```
    def __init__(self, name):
        self.name = name # 부모의 속성 생성 로직
```

--- 학생들의 혼한 당혹감 ---

```
g = GradStudent("Kim", "AI Lab")
```

class GradStudent(Student):

```
    def __init__(self, name, lab):
        # 인자 name은 받았지만...
        # 부모의 __init__은 무시된다.
        self.lab = lab
```

```
print(g.lab) # "AI Lab" (출력 성공)
```

```
print(g.name) # # AttributeError: 'GradStudent'
object has no attribute 'name'
```

```
# has no attribute 'name'
```

상속의 대원칙

IS-A 관계 : 자식 클래스가 부모 클래스의 일종이어야 한다

두 클래스 사이에 “~은 ~의 일종이다”라는 논리가 성립할 때만 사용해야 한다.

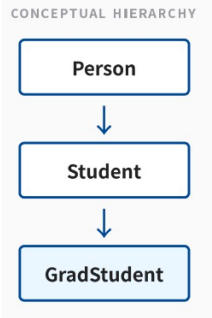
e.g. 대학원생은 학생이다

개는 동물이다

노트북은 컴퓨터다

리스코프 치환 원칙 : 부모가 들어갈 자리에 자식을 넣어도 말이 되어야 한다.

HAS-A 관계에서는 쓰면 안 된다. e.g. 유학생은 전공이다.



상속을 쓰는 이유

재사용성 : 부모 클래스에서 작성된 메서드와 속성을 자식 클래스에서 그대로 물려받아 사용

유지보수의 편리함 : 공통 로직이 부모 클래스 한 곳에만 존재.

부모에서 수정하면 자식에서도 같이 수정됨. **중복 없음.**

계층 구조의 논리적 표현 : 현실 세계의 IS-A 구조 코드로 자연스럽게 구현

IS-A vs HAS-A 관계의 비교 : IS-A에서는 상속 상용. HAS-A에서는 상속 대신 클래스 내부 속성으로 포함

Before: Missing Name Attribute

class Student:

def __init__(self, name):

self.name = name # 부모의 속성 생성 로직

class GradStudent(Student):

def __init__(self, name, lab):

부모의 __init__이 실행되지 않음

self.lab = lab

g = GradStudent("Kim", "AI Lab")

print(g.lab) # "AI Lab" (출력 성공)

print(g.name) # AttributeError 발생!

After: super() Fix

class Student:

def __init__(self, name):

self.name = name

class GradStudent(Student):

def __init__(self, name, lab):

부모 생성자를 명시적으로 호출

super().__init__(name) # 부모 속성 먼저 초기화

self.lab = lab # 그 다음 자식 속성 추가

g = GradStudent("Kim", "AI Lab")

print(g.lab) # "AI Lab" (출력 성공)

print(g.name) # "Kim" (성공!)

자식 클래스에서 __init__을 재정의하면 부모의 초기화 로직은 생략됨.

super()을 사용하여 명시적으로 부모가 설계한 속성들은 현재 객체(self)에 연결해야 함.

★super()뒤에는 self를 절대 넣으면 안 된다!★ 이미 상위 클래스 계층으로 도달했기 때문.

부모 자원의 활용

class Student:

species = "Human" # 클래스 변수

def __init__(self, name):

self.name = name # 인스턴스 속성

def study(self):

print(f"{self.name} 공부 중")

def introduce(self):

print(self.species) # ① 부모의 클래스 변수 접근

print(self.name) # ② 부모의 인스턴스 속성 접근

self.study() # ③ 부모의 메서드 호출

class GradStudent(Student):

def __init__(self, name, lab):

super().__init__(name)

self 없이 호출!

super().__init__(self, name) -> 흔한 실수!

self.lab = lab

g = GradStudent("Alice", "AI Lab")

g.introduce()

- ① self.species : 공유 속성
- ② self.name : 인스턴스 속성
- ③ self.study() : 인스턴스 메서드

클래스 연습

부모 클래스

```
class Student:
    species = "Human"
    def __init__(self, name):
        self.name = name
    def study(self):
        print(f"{self.name} 공부 중")
```

자식 클래스

볼드친 부분 빈칸뚫기

```
class ExchangeStudent(Student): # 1. 부모 클래스 상속
    def __init__(self, name, country):
        super().__init__(name) # 2. 부모 생성자 호출 (self 생략)
        self.country = country # 3. 자식 고유 속성 저장

    def introduce(self):
        print(f"안녕하세요, {self.name}입니다.") # 4. 부모에게 물려받은 속성
        print(f"출신 국가: {self.country}") # 5. 자식 고유 속성
        self.study() # 6. 부모 메서드 호출
```

상속과 Private 속성

#접근 시도 실패

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.__score = score # private 변수

class GradStudent(Student):
    def __init__(self, name, score, lab):
        super().__init__(name, score)
        self.lab = lab

    def show_score(self):
        print(self.__score)
        # -> _GradStudent__score 로 맵글링됨
        # -> _Student__score 와 완전히 다른 변수!

g = GradStudent("Alice", 90, "AI Lab")
g.show_score() # AttributeError 발생!
```

#getter를 통한 접근 - 성공

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.__score = score

    def get_score(self): # getter 메서드 제공
        return self.__score

class GradStudent(Student):
    def __init__(self, name, score, lab):
        super().__init__(name, score)
        self.lab = lab

    def show_score(self):
        print(self.get_score()) #공식 통로를 통해 접근

g = GradStudent("Alice", 90, "AI Lab")
g.show_score() # 출력: 90
```

★Attribute Error가 발생하는 이유: 네임 맵글링 때문이다.

Student 클래스 안의 __score는 내부적으로 _Student__score가 된다.

따라서 자식인 GradStudent 클래스 안의 show_score에서 호출한 self.__score는 파이썬이 _GradStudent__score로 해석해 버린다.

메서드 오버라이딩

: 이름이 완전히 이름이 같고, 동작의 구체화를 하기 위해서 사용.

의도적으로 바꾸기 위해서 사용

```

class Student:
    def study(self):
        print("공부를 합니다.")

class GradStudent(Student):
    # 부모 메서드를 같은 이름으로 다시 구현
    def study(self):
        print("논문을 읽으며 연구합니다.")

g = GradStudent()
g.study() # 호출 시 자식의 메서드가 우선 실행

```

오버라이딩의 문법과 규칙

```

# 부모 클래스: 일반적인 기능 정의
class Student:
    def __init__(self, name):
        self.name = name

    def study(self):
        print(f'{self.name}' 학생이 교재를 읽습니
다.")

# 자식 클래스: 특수한 기능으로 재정의
class GradStudent(Student):
    def __init__(self, name, lab):
        super().__init__(name)
        self.lab = lab

    def study(self): # 오버라이딩!
        print(f'{self.name}' 대학원생이 {self.lab}에서
연구합니다.")

s1 = Student("철수")
s2 = GradStudent("영희", "AI Lab")

s1.study() # 출력: '철수' 학생이 교재를 읽습니다.
s2.study() # 출력: '영희' 대학원생이 AI Lab에서 연
구합니다. (자식 메서드 우선)

```

규칙 1 : 같은 이름. 이름이 정확히 일치해야 한다.
규칙 2 : 자동적용(MRO) 자식 클래스 우선 적용. 자식이 있으면 부모 것은 무시(Shadowing)
규칙 3 : 확장성(super) 부모 메서드를 완전히 대체하지 않고, 일부만 추가하고 싶을 때는 super()를 사용.

완전 재정의 vs super()로 확장

#완전 재정의 class GradStudent(Student): def study(self): # 부모 동작 완전히 무시 print(f'{self.name}: 논문 작성 중") print("문헌 조사, 실험, 집필...")	#super() 확장 class GradStudent(Student): def study(self): super().study() # 부모 동작 실행 print("+ 논문 작성 과정 추가") # 부모 로직을 재활용하여 코드 중복 방지
--	--

완전 재정의 : 부모 클래스로부터 물려받은 기본 동작이 자식 클래스에서는 아무런 의미가 없거나 완전히 결이 다를 때

super() 확장 : 부모가 수행하는 기본 동작을 바탕으로 삼고, 거기에 자식 클래스만의 특수한 절차나 기능을 추가 (Add-on)하고 싶을 때 사용

e.g. 오버라이딩 예제

```
class SmartDevice:
    def turn_on(self):
        print("시스템 전원을 연결합니다.")

class SmartBulb(SmartDevice):
    def turn_on(self):
        # 미션 1: 부모 호출 없이 완전 재정의
        print("조명이 100% 밝기로 켜집니다.")

class SmartSpeaker(SmartDevice):
    def turn_on(self):
        # 미션 2: super()를 이용해 부모의 전원
        # 연결을 먼저 수행
        super().turn_on()
        print("오늘의 뉴스를 브리핑합니다.")
```

```
class SmartTV(SmartDevice):
    def turn_on(self):
        # 미션 3: 확장 전략 선택
        super().turn_on()
        print("지난번 시청하신 채널로 시작합니다.")

# --- 객체 생성 및 다형성 테스트 ---
devices = [SmartBulb(), SmartSpeaker(), SmartTV()]
for d in devices:
    d.turn_on()
```

오버라이딩 vs 오버로딩

구분	메서드 오버라이딩 (Overriding)	메서드 오버로딩 (Overloading)
핵심 개념	부모의 동작을 자식이 재정의	동일 이름에 다양한 파라미터 정의
상속 필수여부	YES (상속 관계 필수)	NO (단일 클래스 내 가능)
파이썬 지원	완전 지원 (다형성의 도구)	미지원 (이름표는 하나)
구현 및 대안	내용 교체 또는 <code>super()</code> 확장	디폴트 인자, <code>*args</code> / <code>**kwargs</code>

```
void study() { ... }
void study(String subject) { ... }
// 시그니처가 다르면 별개 존재로 공존
```

```
def study(self): ...
def study(self, subject): ...
# 마지막 정의가 이름표(Name)를 독점
```

다형성(Polymorphism)

```
devices = [SmartBulb(), SmartSpeaker(), SmartTV()]
for d in devices:
    d.turn_on()

# SmartBulb -> "조명이 100% 밝기로 켜집니다."
# SmartSpeaker -> "시스템 전원을 연결합니다."
# "오늘의 뉴스를 브리핑합니다."
# SmartTV -> (자유 구현 결과)
```

d가 무엇인지 몰라도 — turn_on() 한 줄로 각 기기가 알아서 다르게 동작한다

상속 기반 다형성 : 부모 클래스의 메서드를 자식들이 각자 방식을 **오버라이딩**함으로써, 동일한 호출에 대해 객체마다 고유한 방식으로 응답하는 능력.

- 다형성을 쓰는 이유 : 1. 코드가 간결화. 타입별 if-elif 분기문이 사라진다.
 2. 유연한 확장성. 새 기기(자식 클래스)가 추가되어도 for 루프는 수정할 필요가 없음.

상속 없는 다형성 - Duck Typing

SmartDevice와 아무 상속 관계 없음

```
class OldRadio:
    def turn_on(self):
        print("Radio: 지직... 전파를 수신합니다.")

devices = [
    SmartBulb(),
    SmartSpeaker(),
    OldRadio() # 상속 없이 리스트에 합류
]
```

```
for d in devices:
    d.turn_on() # turn_on()이 있으면 그냥 실행
```

오리처럼 걷고 오리처럼 소리 내면 오리다.

OldRadio 클래스는 SmartDevice를 전혀 상속받지 않은 독립적인 클래스다.

하지만 내부에 turn_on() 메서드를 가지고 있다.

-> 파이썬은 상속을 받았는지 안받았는지 따지지 않음. 같이 묶여서 for 루프 안에서 실행될 수 있음.

★Duck typing은 IS-A가 아니라, CAN-DO를 기준으로 판단한다.

object와 MRO

```
class SmartDevice: pass
class SmartTV(SmartDevice): pass

# __mro__ 속성으로 탐색 순서 확인
print(SmartTV.__mro__)
```

```
# 출력 결과 (Tuple):
(
    <class '__main__.SmartTV'>,
    <class '__main__.SmartDevice'>,
    <class 'object'> # 마지막은 항상 object
)
```

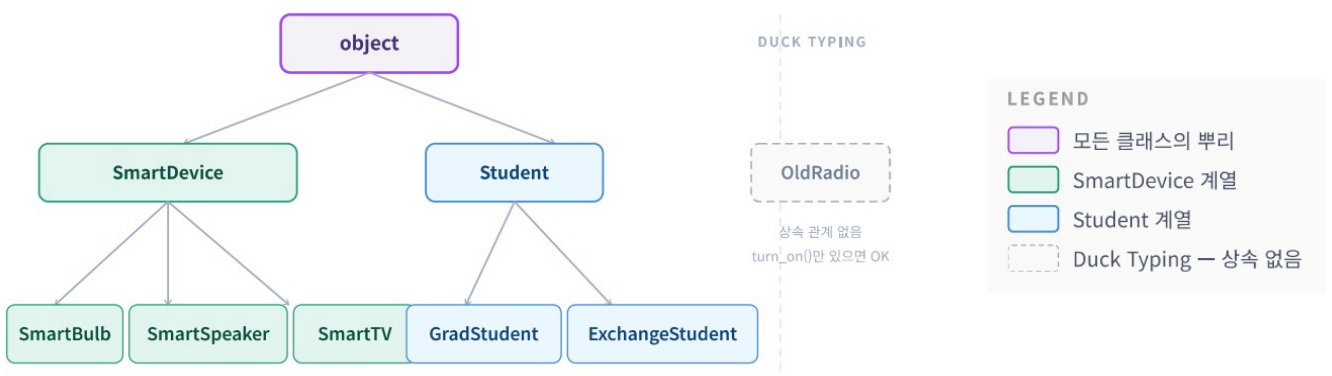
모든 클래스의 조상 "object". 파이썬에서 계층 구조를 끝까지 타고 올라가면 맨 꼭대기에 object라고 하는 루트 클래스가 자동으로 상속되어 자리 잡고 있다.

__mro__ : 파이썬이 메서드를 탐색하는 순서 볼 수 있음.

object

```
print(SmartTV.__mro__)
# 출력 결과: (<class '__main__.SmartTV'>, <class '__main__.SmartDevice'>, <class 'object'>)
```

```
print(GradStudent.__mro__)
# 출력 결과: (<class '__main__.GradStudent'>, <class '__main__.Student'>, <class 'object'>)
```



상속 계층의 코드 검증

```
# 1. isinstance: 객체의 가문(Family) 확인
g = GradStudent()
print(isinstance(g, Student))    # True (부모 인정)
print(isinstance(g, object))     # True (조상 인정)
```

```
# 2. issubclass: 클래스 간 설계 관계 확인
print(issubclass(GradStudent, Student)) # True

# 3. type()과의 결정적 차이
print(type(g) == Student)    # False (직계만 인정)
```

type() : 정확히 해당하는 클래스인가?

isinstance() : g가 Student가문에 포함되니?

issubclass() : GradStudent가 Student의 자식이 맞나?

e.g. 다형성을 활용한 통합제어 시스템

```
class SmartDevice:
    pass

class SmartPlug(SmartDevice):
    def turn_off(self):
        print("플러그 전원을 차단합니다.")

class OldRadio:
    # 족보 없음 (SmartDevice를 상속받지 않음)
    def turn_off(self):
        print("라디오 회로를 끄습니다.")

def manage_system(target):
    # [빈칸 1] target이 SmartDevice 가문인지 계층
    # 까지 추적하여 확인

    if isinstance(target, SmartDevice):
        print("스마트 기기 확인됨")

    # [빈칸 2] 상속 여부와 상관없이 객체 내부에
    # turn_off 기능이 있는지 확인
    if hasattr(target, "turn_off"):
        target.turn_off()

# --- 통합 제어 실행 ---
inventory = [SmartPlug(), OldRadio()]
for item in inventory:
    manage_system(item)
```

Lecture 10 : Git & Github

버전관리의 필요성

과거 이력 추적 불가 : 파일 이름만 바뀌서는 정확히 어떤 부분이 바뀌었는지 알 수 없음

협업 충돌 발생 : 여러 팀원이 하나의 프로젝트를 동시에 수정하여 폴더에 덮어쓰기 시작하면 코드가 꼬이고 유실

-> **버전 관리 시스템(VCS)**은 변경 이력을 시스템이 자동으로 시간대별로 기록해준다.

에러가 발생하면 언제든지 안전했던 이전 버전의 상태로 타임머신을 타듯 되돌릴 수 있다.

충돌을 체계적으로 조율해준다.

Git이란 무엇인가?

: Git은 파일의 변경 이력을 추적하고 관리하는 **분산 버전 관리 시스템(VCS)**

로컬에서 인터넷 없이도 완전히 동작.

Git의 핵심 특징 4가지

로컬 중심 동작 : 오프라인 환경에서 commit, 이력 조회, 브랜치 작업 등이 모두 가능

스냅샷 방식 : 단순히 파일의 바뀐 차이점(diff)만 구분하는 방식이 아니라 매 commit마다 프로젝트 전체 상태의 스냅샷을 저장

빠른 속도 : 대부분의 작업이 로컬에서 이루어짐. 속도가 매우 빠름

무결성 보장 : 모든 데이터는 SHA-1로 관리되어 변조 여부를 바로 알 수 있음.

-> 2005년 리누스 토르발즈가 만든 오픈소스 프로젝트. 현재 개발자의 90%이상이 사용. 표준 도구임.

Git과 GitHub

구분	Git	GitHub
정체	버전 관리 소프트웨어 (도구)	Git 저장소를 호스팅하는 웹 서비스
위치	내 컴퓨터(로컬)에 설치	인터넷 클라우드 서버
인터넷	없어도 동작 (오프라인 가능)	인터넷 필수
주요 기능	commit, branch, merge, log...	코드 공유, Pull Request, 협업, CI/CD...
비유	📷 사진을 찍는 카메라	☁ 사진을 올리는 클라우드

Repository

: 프로젝트의 모든 파일과 변경 이력 전체를 담고 있는 저장소.
내부에 숨겨진 .git 폴더가 모든 이력을 관리함.

Local Repository : 내 컴퓨터에 있는 저장소. .git 폴더가 있는 디렉토리가 곧 로컬 repo.
인터넷 없이도 모든 Git 작업이 가능함.

Remote Repository : GitHub 서버에 있는 저장소.
github.com/username/repo-name 형태의 URL을 갖음.
팀원과 코드를 공유하는 공간

★로컬과 원격은 독립적으로 존재함. push와 pull을 통해 서로 동기화함.

Git의 세 가지 영역



Working Directory -- git add -> Staging Area --git commit-> Local Repo --git push-> Remote Repo

Staging Area의 필요성 : 수정한 파일이 10개여도, 그 중 관련 있는 것만 골라서 하나의 commit으로 묶을 수 있기 때문. commit을 의미 있는 단위로 만들 수 있는 것이 Git의 강점

Commit

: Staging Area에 올라온 변경 사항을 로컬 저장소에 영구 기록하는 행위. 각 commit은 고유한 ID(해시값)을 가지며, 작성자·시간·메시지가 함께 저장된다.

❌ 나쁜 예시 (무엇을 했는지 알 수 없음)

"수정"
"ㅇㅇ"
"asdf"
"final"
"코드 고침"

● 좋은 예시 (무엇을 왜 바꿨는지 명확함)

"Add student grade calculator function"
"Fix division by zero error in average()"
"Refactor file reading with pathlib"
"Add .gitignore for Python project"

Push & Pull

git push	로컬 커밋을 원격으로 업로드
git pull	원격의 최신 이력을 내 컴퓨터로 다운로드, 자동으로 합치기
git clone	원격 저장소 프로젝트를 처음으로 내 컴퓨터에 통째로 복사
git fetch	원격 저장소의 변경 사항을 로컬로 가져오되, 자동으로 코드를 합치지는 않음.

commit을 했다고 GitHub에 올라간 것이 아님! 반드시 push를 해야 GitHub에 반영됨.

Lecture 11 : Branch & CLI

브랜치

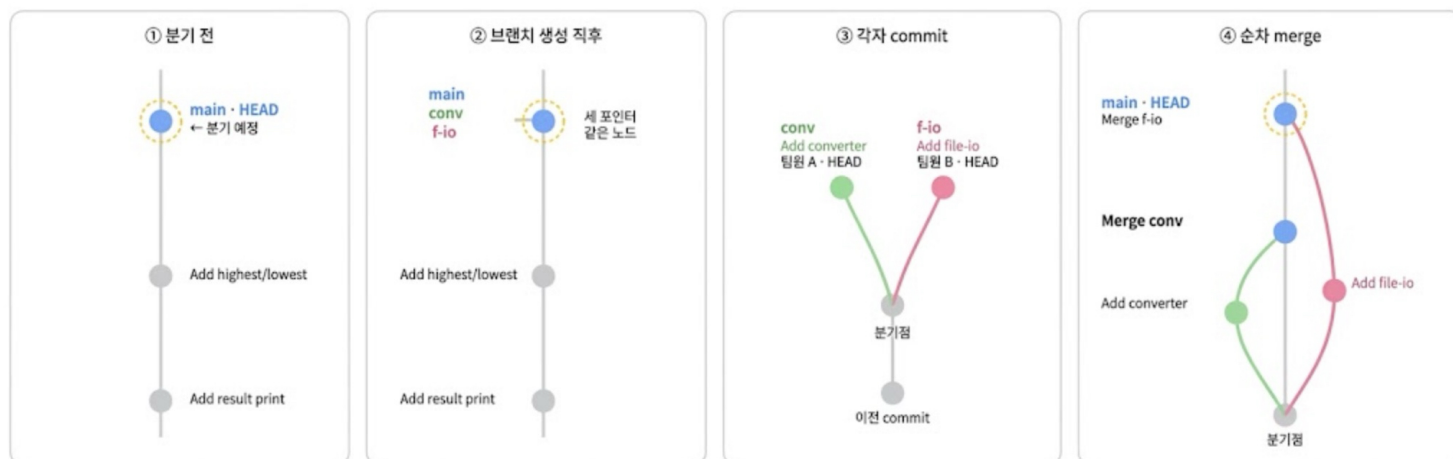
- : 특정 commit을 가리키는 포인터
- 파일을 복사하는 게 아니라 **작업 흐름을 분기하는 것**

브랜치의 필요성

브랜치 없이 MAIN에서 바로 작업하면 기능 개발 중 버그가 생겨도 main에 불완전한 코드가 그대로 남는다. 여러 사람이 동시에 같은 파일을 수정하면 충돌이 걸잡을 수 없이 커진다.

- main 브랜치** : 항상 동작하는 안정적인 상태 유지. 완성된 기능만 여기에 합치기
- feature 브랜치** : 새 기능은 별도 브랜치에서 개발. 실패해도 main에는 영향이 없음.
- merge** : 기능이 완성되면 feature 브랜치를 main에 합침.

HEAD와 브랜치 그래프



HEAD : 현재 내가 작업 중인 위치를 가리키는 포인터.

브랜치를 전환하면 HEAD가 그 브랜치로 이동

CLI 입문

서버에는 GUI가 없음. 자동화 스크립트에서 Git 명령어를 직접 씁니다.
에러 메시지가 터미널에 출력됩니다.

Unstaged 파일 확인

git status

현재 변경된 파일 목록 확인

파일을 **Staged**로 이동

git add 파일명

특정 파일을 Staging Area에 추가

전체 파일 **Staged**로 이동

git add .

변경된 모든 파일을 한 번에 추가

Commit 버튼

git commit -m "메시지"

스냅샷을 로컬 저장소에 기록

Push 버튼	<code>git push</code>	로컬 commit을 원격에 업로드
Pull 버튼	<code>git pull</code>	원격 변경사항을 로컬로 내려받기
커밋 그래프 보기	<code>git log --oneline --graph</code>	commit 이력을 텍스트 그래프로 확인

브랜치 관련 명령어

브랜치 목록 확인	<code>git branch</code>	로컬 브랜치 목록. 현재 브랜치에 * 표시
브랜치 생성	<code>git branch feature/이름</code>	새 브랜치 생성 (전환은 하지 않음)
브랜치 전환	<code>git switch feature/이름</code>	HEAD를 해당 브랜치로 이동
생성 + 전환 동시에	<code>git switch -c feature/이름</code>	가장 자주 쓰는 방식
main에 merge	<code>git merge feature/이름</code>	현재 브랜치에 대상 브랜치를 합침
브랜치 삭제	<code>git branch -d feature/이름</code>	merge 완료 후 정리

GitHub Flow

: 단순하고 직관적인 브랜치 전략. main+feature 브랜치 두 종류만 사용.

1. 브랜치 생성 main에서 feature 브랜치 만듦.
이름은 기능을 설명하도록 지음. feature/grade-converter
2. 작업 + commit feature 브랜치에서 기능을 개발하고 commit을 쌓음. main은 건드리지 않음.
3. Push + PR feature 브랜치를 원격에 push하고 Pull Request를 만듦.
4. 리뷰 + Merge 팀원이 코드를 검토하고, main에 merge함.
5. 브랜치 삭제 merge 완료된 feature 브랜치는 삭제하여 깔끔하게 유지.

Lecture 12 : Pull Request & Merge Conflict

Git 충돌의 발생 원인

두 사람이 같은 파일의 같은 위치를 서로 다르게 수정함.

A : "학점 변환" 항목 추가

B : "파일 저장" 항목 추가

-> Git이 어느 것을 선택해야 할지 모름 ==> Conflict!

충돌이 왜 생길까? 충돌 마커는 어떻게 읽지? VS Code에서 해결하는 방법. PR로 사전에 조율

Merge Conflict

: 두 브랜치가 같은 파일의 같은 줄을 서로 다르게 수정했을 때, Git이 자동으로 합치지 못하고 사람의 판단을 요청하는 상태.

1. Git이 자동으로 합치는 경우 : 두 브랜치가 다른 파일을 수정 or 두 브랜치가 같은 파일의 다른 줄을 수정
-> 자동 merge 성공
2. Conflict가 발생하는 경우 : 두 브랜치가 같은 파일의 같은 줄 또는 인접한 줄을 서로 다르게 수정한 경우
-> Git이 판단 불가 -> Conflict 발생

가장 흔한 Conflict 상황 : readme.md처럼 여러 사람이 동시에 수정하는 공유 파일에서 자주 발생.

pull없이 작업하다가 merge할 때 뒤늦게 충돌을 발견하는 경우가 가장 많음.

Conflict 마커

: Git이 충돌한 두 내용을 나란히 보여주며 "어느 것을 쓸지 결정하세요"라고 알림.

기능 - 과목별 점수 입력 - 평균 점수 계산

```
<<<<<<< HEAD                                // 현재 내가 있는 브랜치. 여기서부터 ==== 까지
- 점수 파일 저장 / 불러오기
=====
- 점수 -> 학점 변환 (A+~F)                  // 합치려는 브랜치(main)의 내용
>>>>>>> main
```

Conflict 해결 원칙

git switch main

git merge feature/file-io

-> main에서 conflict를 해결하면 안된다!! main이 일시적으로 불안정. 다른 팀원에게 영향을 줌

git switch feature/file-io

git merge main

-> feature 브랜치에서 conflict를 해결해야 함. main은 건드리지 않음.

해결 완료 후 main에 merge하기

★충돌 해결법★

feature 브랜치로 전환	// 작업 중이던 독립된 공간으로 이동
-> git merge main	// 최신 main 브랜치의 내용을 끌고와 merge 시도. Conflict 발생!
-> VS 코드에서 conflict 해결	// 충돌 마커를 확인하면서 코드를 안전하게 수정
-> git add + commit	// 수정사항 저장
-> push	
-> main에 merge	// 깨끗해진 코드를 main으로 최종 병합

Pull Request(PR)

내가 독립된 브랜치에서 기능을 다 만들었는데 main 브랜치에 합쳐도 될까요? 하고 팀원에게 요청하는 GitHub기능 merge전에 코드를 검토하고 conflict를 사전에 감지. **게이트키퍼 역할**

코드 리뷰 : 팀원들이 변경된 소스코드를 라인 단위로 꼼꼼하게 검토하며 의견을 교환할 수 있음.

Conflict 사전 감지 : 머지 단추를 누르기 전에, GitHub이 두 브랜치 간에 충돌이 발생하는지 자동으로 체크

Approve 후 Merge : 리뷰어가 approve해야만 merge가 가능. main에는 항상 검증된 코드만 들어옴.

PR은 Fork 없이도 사용 가능

같은 repo에서 feature 브랜치 -> main으로 PR을 만드는 것이 팀 협업의 표준 방식.

Fork는 외부 기여자가 권한 없이 기여할 때 사용.

PR 워크플로 전체 흐름

STEP 1 [브랜치 생성]: main에서 feature 브랜치 분기

STEP 2 [작업 + commit]: feature 브랜치에서 개발

STEP 3 [PR 생성]: GitHub에서 리뷰 요청

STEP 4 [리뷰 + 합의]: Conversation으로 조율 // 합의가 핵심이다. GitHub PR Conversation에서 대화를 나눔.

STEP 5 [Conflict 해결]: feature 브랜치에서 먼저

STEP 6 [Approve + Merge]: 리뷰어 승인 후 merge

Branch Ruleset: PR + Approve 없이 main에 직접 merge/push 불가. main을 항상 안정적으로 유지하는 장치.

import calculator	# ① 모듈 전체 가져오기
calculator.add(3, 5)	# 모듈명.함수명
from calculator import add	# ② 특정 함수만 가져오기
add(3, 5)	# 함수명만 사용
from calculator import add, subtract	# ③ 여러 함수 한꺼번에
import calculator as calc	# ④ 별칭(alias) 사용
calc.add(3, 5)	# 짧게 줄여서
* 다른 패키지에서 같은 이름 메서드(함수) import 하면 충돌이 난다. 뉘어씩워짐	
* from 패키지 import *	#하지 말아라! 네임스페이스 오염

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

# 이 블록: 직접 실행할 때만 동작	# import될 때는 실행되지 않음
if __name__ == "__main__":	
print(add(3, 5))	# 테스트
print(subtract(9, 4))	# 테스트

my_project/	# main.py 에서
├── main.py	from mytools.calculator import add
└── mytools/	from mytools import file_utils
├── __init__.py	
├── calculator.py	result = add(3, 5)
└── file_utils.py	file_utils.read_csv('data.csv')

_____이터레이터_____

엔진 하부 내부 동작 코드 추적

fruits = ['apple', 'banana']



```
it = iter(fruits) # __iter__() 구동 (배달원 고용)
print(next(it))   # 'apple' 반환 (첫 번째 물건 꺼내기)
print(next(it))   # 'banana' 반환 (두 번째 물건 꺼내기)
print(next(it))   # StopIteration 예외 발생! (더 꺼낼 게 없음!)
```

_____제너레이터와 지연 평가_____

```
# 리스트 컴프리헨션. 100만개 인스턴스가 동시에 올라가 8MB 점유.100만 개의 데이터를 즉각적으로 계산 후 할당
numbers = [x * 2 for x in range(1_000_000)]
print(numbers[0])
```

이 방식은 제너레이터 함수형. 요구 시 순차 산출. yield 문을 활용한 지연 양보 처리

```
def gen_numbers(n):
    for x in range(n):
        yield x * 2 # 값을 외부로 전달 후 일시정지
```

Lazy Evaluation 지연평가 : 실제 데이터 연산의 실행 시점을 소비자가 요구하는 그 순간까지 미루는 최적화 패러다임